

Medicine inventory management system

Fundamentals of Database

Cbio204

Name	ID
Lamar Ahmed Refaat	241001538
Asmaa Mostafa	241002393
Asmaa Walid	241002055
Rawan Hossam	241002259

Introduction:

This database project presents Medicine Inventory Management System designed to streamline the operations of a healthcare facility's pharmacy and prescription management processes. The system efficiently handles the storage and retrieval of information related to doctors, diseases, medicines, prescriptions and billing.

Description:

Database Implementation: DDL & DML

Overview

This section presents the complete database implementation of the Medicine Inventory System, divided into two phases: the structural definition of the database using DDL (Data Definition Language), and the initial data population using DML (Data Manipulation Language). Together, these scripts form the foundation of a fully functional relational database designed to manage doctors, diseases, medicines, prescriptions, and billing records.

Phase 1 — Database & Table Creation (DDL)

The implementation begins by creating the `medicine_inventory_db` database using the `IF NOT EXISTS` clause to ensure the script can be safely re-executed without errors. The `USE` statement then directs all subsequent operations into that database.

Seven tables are created to represent the core entities and relationships of the system:

The `Doctors` table stores the profiles of medical professionals, capturing their name, academic qualification, and area of specialization. The `Diseases` table maintains a structured catalog of medical conditions, with the disease type restricted to four controlled categories — Infectious, Deficiency, Genetic Hereditary, and Non-Genetic Hereditary — enforced through an `ENUM` constraint to ensure data consistency. The `Medicines` table holds the complete medicine inventory, recording each medicine's name, manufacture and expiry dates, price, and dosage instructions.

To model the many-to-many relationship between medicines and diseases, the `Medicine_Diseases` junction table is introduced, using a composite primary key

formed from both foreign keys to prevent duplicate associations. Cascading deletion is applied on both sides so that removing a medicine or disease automatically cleans up its related mappings.

The Prescriptions table links each prescription to the issuing doctor and records the patient's name alongside the prescription date. The Prescription_Medicines table then breaks down each prescription into individual medicine line items, storing the medicine reference and the prescribed quantity. Finally, the Bills table records the financial details of each prescription, including tax, discount, and total amount. A UNIQUE constraint on the prescription_id column enforces a strict one-to-one relationship, ensuring no prescription is billed more than once. Default values of 0.00 are applied to all monetary fields, allowing bills to be created and calculated in a separate step.

Phase 2 — Data Population (DML)

Following the structural setup, the database is populated with realistic sample data using INSERT statements across all seven tables.

10 doctors are inserted representing a diverse range of specializations including Cardiology, Pulmonology, Endocrinology, Neurology, and Pediatrics, among others. 10 diseases are inserted spanning all four disease type categories, including conditions such as Influenza, Tuberculosis, Sickle Cell Disease, Type 2 Diabetes, and Anemia. 10 medicines are inserted covering a variety of drug types such as Paracetamol, Amoxicillin, Metformin, Insulin, and Vitamin C, each with complete manufacturing and expiry dates, pricing, and dosage instructions.

12 medicine-disease mappings are inserted into the Medicine_Diseases table to establish which medicines treat which conditions. These mappings demonstrate the many-to-many nature of the relationship — for example, Vitamin C is linked to both Scurvy and Anemia, while Ibuprofen is linked to both Hypertension and Asthma.

10 prescriptions are inserted, each issued by a different doctor to a unique patient, with dates spread across the year 2023. Each prescription is then assigned a corresponding bill in the Bills table, with varying tax rates and selective discounts applied to certain prescriptions. Notably, the total field is initialized to 0.00 across all bills, reflecting a design decision to compute and update the final total separately — for instance, through an UPDATE query that factors in the medicines' prices, quantities, tax, and discount.

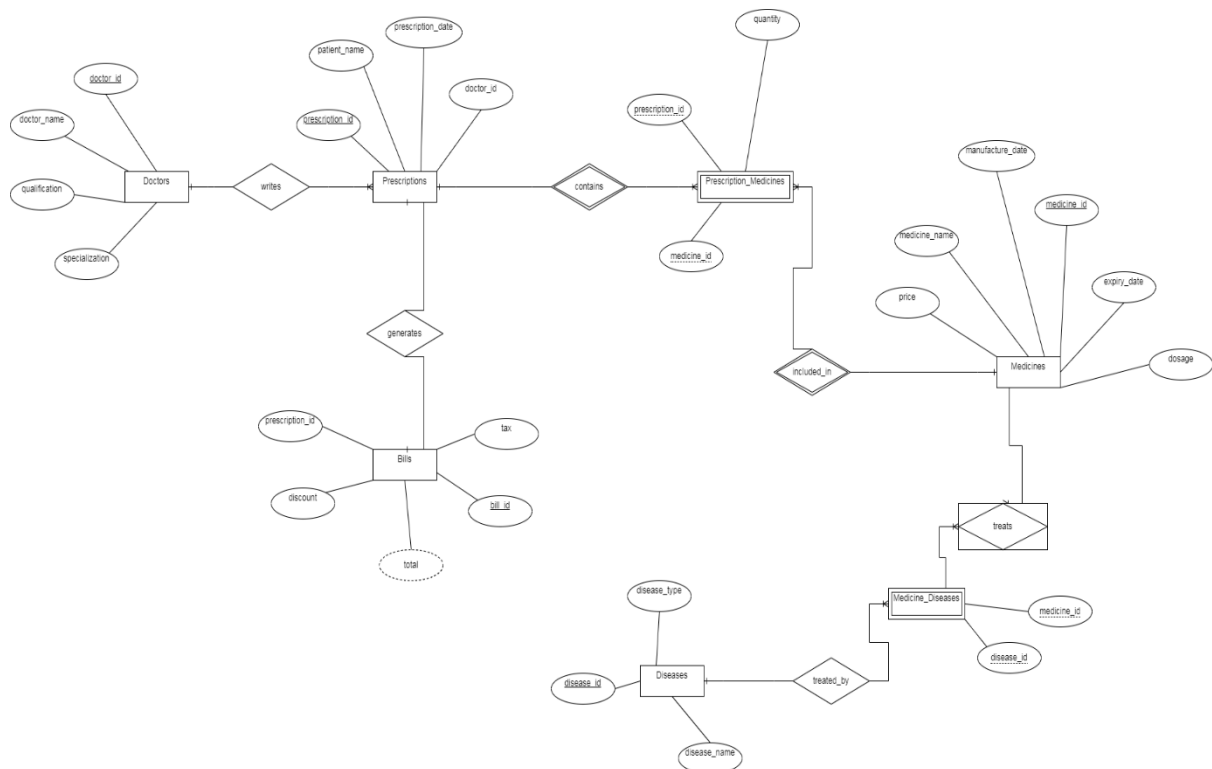
Finally, 20 prescription medicine line items are inserted into the Prescription_Medicines table, assigning exactly two medicines to each prescription, with specified quantities — completing the full data flow from doctor and patient, through prescribed medicines, down to the final bill.

Entities and Attributes:

Entity	Attributes
Doctors	doctor_id, doctor_name, qualification, specialization
Diseases	disease_id, disease_name, disease_type
Medicines	medicine_id, medicine_name, manufacture_date, expiry_date, price, dosage
Medicine_Diseases	medicine_id, disease_id
Prescriptions	prescription_id, doctor_id, patient_name, prescription_date
Prescription_Medicines	prescription_medicine_id, prescription_id, medicine_id, quantity
Bills	bill_id, prescription_id, tax, discount, total

The Doctors table stores doctors who write prescriptions, including their name, qualification, and specialization. The Diseases table maintains information about medical conditions with a name and type classification (Infectious, Deficiency, Genetic Hereditary, Non-Genetic Hereditary). The Medicines table manages the drug inventory including name, manufacture date, expiry date, and dosage instructions. The Medicine_Diseases table is a bridge entity that links medicines to the diseases they treat, establishing a many-to-many relationship. The Prescriptions table records patient prescriptions including which doctor issued it, the patient's name, and the date prescribed. The prescription_Medicines table is a junction table that connects prescriptions to the specific medicines and quantities prescribed. The Bills table handles the financial transactions for each prescription, including tax, discount, and automatically calculated total amount.

ER Diagram:



Relational Schema:

Doctors (doctor_id (pk) , doctor_name , qualification , specialization)

Diseases (disease_id (pk) , disease_name , disease_type)

Medicines (medicine_id (pk) , medicine_name , manufacture_name , expiry_date , price , dosage)

Medicine_Diseases (medicine_id (pk) , disease_id (pk))

Prescriptions (prescription_id (pk) , doctor_id , patient_name , prescription_date)

Prescription_Medicines (prescription_medicine_id (pk) , prescription_id , medicine_id , quantity)

Bills (bill_id (pk) , prescription_id , tax , discount , total)

Foreign Keys:

Medicine_Diseases.medicine_id references Medicines.medicine_id

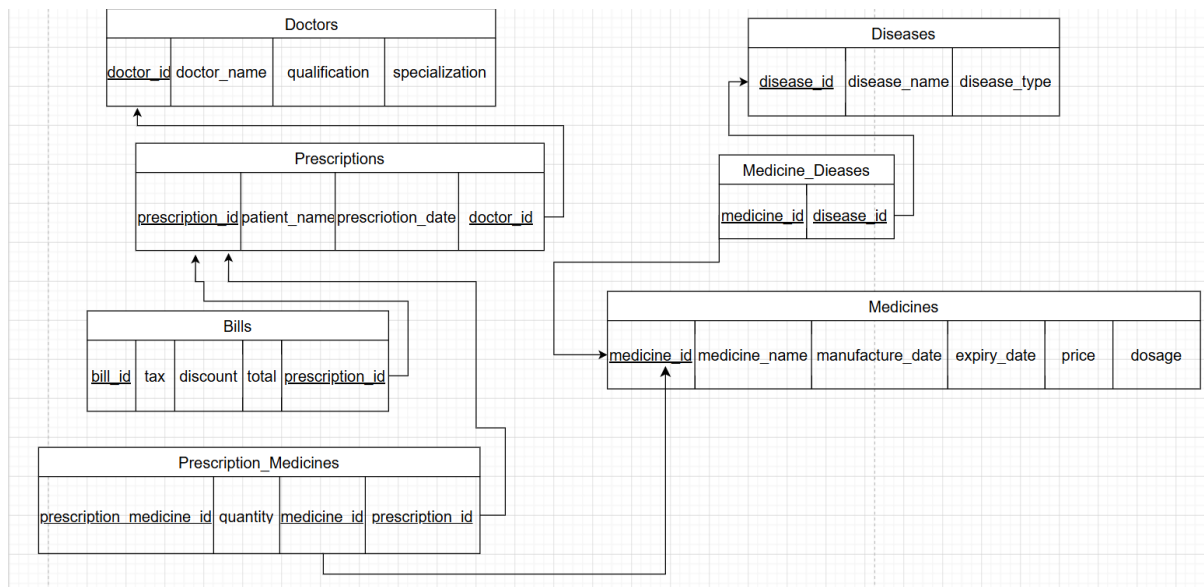
Medicine_Diseases.disease_id references Diseases.disease_id

Prescriptions.doctor_id references Doctors.doctor_id

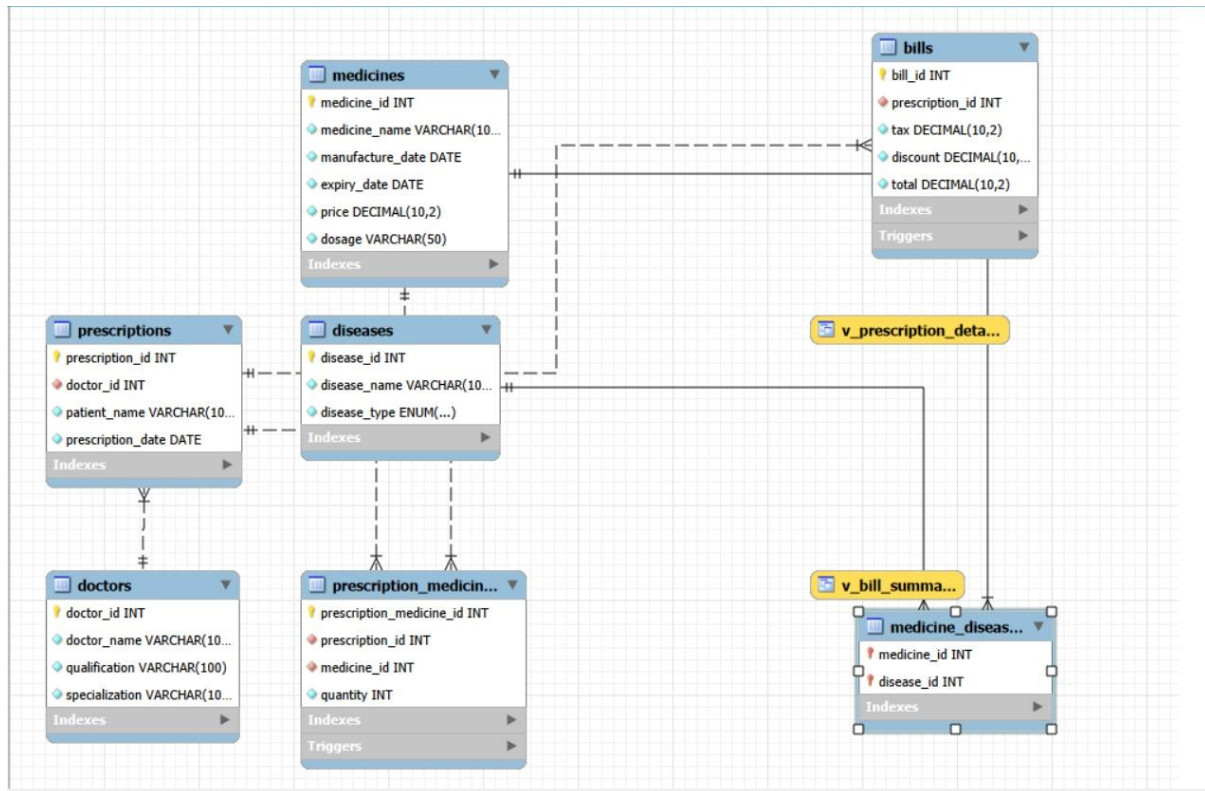
Prescription_Medicines.prescription_id references Prescriptions.prescription_id

Prescription_Medicines.medicine_id references Medicines.medicine_id

Bills.prescription_id references Prescriptions.prescription_id



MySQL Workbench Physical Schema:



Create Tables (DDL):

```
CREATE DATABASE IF NOT EXISTS medicine_inventory_db;
```

```
USE medicine_inventory_db;
```

```
CREATE TABLE Doctors (  
    doctor_id INT AUTO_INCREMENT PRIMARY KEY,  
    doctor_name VARCHAR(100) NOT NULL,  
    qualification VARCHAR(100) NOT NULL,  
    specialization VARCHAR(100) NOT NULL  
);
```

```

CREATE TABLE Diseases (
    disease_id INT AUTO_INCREMENT PRIMARY KEY,
    disease_name VARCHAR(100) NOT NULL,
    disease_type ENUM(
        'Infectious',
        'Deficiency',
        'Genetic Hereditary',
        'Non-Genetic Hereditary'
    ) NOT NULL
);

```

```

CREATE TABLE Medicines (
    medicine_id INT AUTO_INCREMENT PRIMARY KEY,
    medicine_name VARCHAR(100) NOT NULL,
    manufacture_date DATE NOT NULL,
    expiry_date DATE NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    dosage VARCHAR(50) NOT NULL
);

```

```

CREATE TABLE Medicine_Diseases (
    medicine_id INT NOT NULL,
    disease_id INT NOT NULL,
    PRIMARY KEY (medicine_id, disease_id),
    FOREIGN KEY (medicine_id) REFERENCES Medicines(medicine_id) ON DELETE CASCADE,
    FOREIGN KEY (disease_id) REFERENCES Diseases(disease_id) ON DELETE CASCADE
);

```

```

CREATE TABLE Prescriptions (
    prescription_id INT AUTO_INCREMENT PRIMARY KEY,
    doctor_id INT NOT NULL,
    patient_name VARCHAR(100) NOT NULL,
    prescription_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
);

```

);

```
CREATE TABLE Prescription_Medicines (  
    prescription_medicine_id INT AUTO_INCREMENT PRIMARY KEY,  
    prescription_id INT NOT NULL,  
    medicine_id INT NOT NULL,  
    quantity INT NOT NULL,  
    FOREIGN KEY (prescription_id) REFERENCES Prescriptions(prescription_id) ON DELETE CASCADE,  
    FOREIGN KEY (medicine_id) REFERENCES Medicines(medicine_id)  
);
```

```
CREATE TABLE Bills (  
    bill_id INT AUTO_INCREMENT PRIMARY KEY,  
    prescription_id INT NOT NULL UNIQUE,  
    tax DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
    discount DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
    total DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
    FOREIGN KEY (prescription_id) REFERENCES Prescriptions(prescription_id) ON DELETE CASCADE  
);
```

Load Data (DML):

```
USE medicine_inventory_db;
```

```
INSERT INTO Doctors (doctor_name, qualification, specialization) VALUES  
( 'Dr. Aya Hassan', 'MD', 'Heart Disease'),  
( 'Dr. Omar Khaled', 'MD', 'Pulmonology'),  
( 'Dr. Sara Ali', 'PhD', 'Endocrinology'),  
( 'Dr. Hany Mostafa', 'MD', 'Infectious Disease'),  
( 'Dr. Dina Samir', 'MD', 'Gastroenterology'),  
( 'Dr. Laila Tarek', 'MD', 'Neurology'),  
( 'Dr. Youssef Magdy', 'MD', 'Pediatrics'),
```

('Dr. Mona Adel', 'PhD', 'Hematology'),
('Dr. Khaled Nabil', 'MD', 'Dermatology'),
('Dr. Nour Said', 'MD', 'Nephrology');

INSERT INTO Diseases (disease_name, disease_type) VALUES

('Influenza', 'Infectious'),
('Tuberculosis', 'Infectious'),
('Scurvy', 'Deficiency'),
('Rickets', 'Deficiency'),
('Sickle Cell Disease', 'Genetic Hereditary'),
('Hemophilia', 'Genetic Hereditary'),
('Asthma', 'Non-Genetic Hereditary'),
('Hypertension', 'Non-Genetic Hereditary'),
('Type 2 Diabetes', 'Non-Genetic Hereditary'),
('Anemia', 'Deficiency');

INSERT INTO Medicines (medicine_name, manufacture_date, expiry_date, price, dosage) VALUES

('Paracetamol 500mg', '2024-01-10', '2026-01-10', 20.00, '1 tablet twice daily'),
('Amoxicillin 500mg', '2023-08-15', '2025-08-15', 45.00, '1 capsule every 8 hours'),
('Ibuprofen 400mg', '2023-12-20', '2025-12-20', 30.00, '1 tablet after meals'),
('Cough Syrup', '2024-02-05', '2025-06-05', 55.00, '10 ml twice daily'),
('Vitamin C 1000mg', '2024-03-12', '2026-03-12', 25.00, '1 tablet daily'),
('Metformin 500mg', '2023-09-18', '2025-09-18', 60.00, '1 tablet twice daily'),
('Aspirin 81mg', '2024-02-28', '2026-02-28', 18.00, '1 tablet daily'),
('Omeprazole 20mg', '2023-11-02', '2025-11-02', 65.00, '1 capsule daily'),
('Antihistamine 10mg', '2023-07-25', '2025-07-25', 35.00, '1 tablet daily'),
('Insulin 100IU', '2023-05-30', '2025-05-30', 120.00, 'As directed');

INSERT INTO Medicine_Diseases (medicine_id, disease_id) VALUES

(1, 1),
(2, 2),
(3, 8),
(4, 1),
(5, 3),

(5, 10),
(6, 9),
(7, 8),
(8, 7),
(9, 1),
(10, 9),
(3, 7);

INSERT INTO Prescriptions (doctor_id, patient_name, prescription_date) VALUES

(1, 'Ahmed Hassan', '2023-01-10'),
(2, 'Mona Adel', '2023-02-14'),
(3, 'Sara Ali', '2023-03-20'),
(4, 'Omar Khaled', '2023-04-05'),
(5, 'Nour Said', '2023-05-11'),
(6, 'Hany Mostafa', '2023-06-22'),
(7, 'Dina Samir', '2023-07-15'),
(8, 'Youssef Magdy', '2023-08-19'),
(9, 'Laila Tarek', '2023-09-03'),
(10, 'Khaled Nabil', '2023-10-27');

INSERT INTO Bills (prescription_id, tax, discount, total) VALUES

(1, 3.25, 0.00, 0.00),
(2, 5.00, 0.00, 0.00),
(3, 4.75, 0.00, 0.00),
(4, 6.25, 5.00, 0.00),
(5, 2.80, 0.00, 0.00),
(6, 7.25, 10.00, 0.00),
(7, 3.75, 0.00, 0.00),
(8, 6.25, 0.00, 0.00),
(9, 6.25, 5.00, 0.00),
(10, 2.90, 0.00, 0.00);

INSERT INTO Prescription_Medicines (prescription_id, medicine_id, quantity) VALUES

(1, 1, 2),

(1, 5, 1),
(2, 2, 1),
(2, 4, 1),
(3, 3, 2),
(3, 9, 1),
(4, 6, 1),
(4, 8, 1),
(5, 7, 2),
(5, 1, 1),
(6, 10, 1),
(6, 5, 1),
(7, 2, 1),
(7, 3, 1),
(8, 4, 1),
(8, 9, 2),
(9, 8, 1),
(9, 6, 1),
(10, 1, 2),
(10, 7, 1);

Views:

USE medicine_inventory_db;

CREATE OR REPLACE VIEW v_prescription_details AS

SELECT pr.prescription_id,
 pr.prescription_date,
 pr.patient_name,
 d.doctor_name,
 m.medicine_name,
 pm.quantity,
 m.dosage,
 m.price

```

FROM Prescriptions pr
JOIN Doctors d ON pr.doctor_id = d.doctor_id
JOIN Prescription_Medicines pm ON pr.prescription_id = pm.prescription_id
JOIN Medicines m ON pm.medicine_id = m.medicine_id;

CREATE OR REPLACE VIEW v_bill_summary AS
SELECT b.bill_id, b.prescription_id, pr.patient_name, d.doctor_name, b.tax, b.discount, b.total
FROM Bills b
JOIN Prescriptions pr ON b.prescription_id = pr.prescription_id
JOIN Doctors d ON pr.doctor_id = d.doctor_id;

```

Triggers:

```

USE medicine_inventory_db;

DELIMITER //

CREATE TRIGGER bills_before_insert
BEFORE INSERT ON Bills
FOR EACH ROW
BEGIN
    SET NEW.total = (
        SELECT IFNULL(SUM(pm.quantity * m.price), 0)
        FROM Prescription_Medicines pm
        JOIN Medicines m ON m.medicine_id = pm.medicine_id
        WHERE pm.prescription_id = NEW.prescription_id
    ) + NEW.tax - NEW.discount;
END//

```

```

CREATE TRIGGER bills_before_update
BEFORE UPDATE ON Bills
FOR EACH ROW
BEGIN
    SET NEW.total = (
        SELECT IFNULL(SUM(pm.quantity * m.price), 0)
        FROM Prescription_Medicines pm
        JOIN Medicines m ON m.medicine_id = pm.medicine_id
        WHERE pm.prescription_id = NEW.prescription_id
    ) + NEW.tax - NEW.discount;
END//

```

```

CREATE TRIGGER prescription_medicine_after_insert
AFTER INSERT ON Prescription_Medicines
FOR EACH ROW
BEGIN
    UPDATE Bills
    SET total = (
        SELECT IFNULL(SUM(pm.quantity * m.price), 0)
        FROM Prescription_Medicines pm
        JOIN Medicines m ON m.medicine_id = pm.medicine_id
        WHERE pm.prescription_id = NEW.prescription_id
    ) + tax - discount
    WHERE prescription_id = NEW.prescription_id;
END//

```

```

CREATE TRIGGER prescription_medicine_after_update
AFTER UPDATE ON Prescription_Medicines
FOR EACH ROW
BEGIN
    UPDATE Bills
    SET total = (
        SELECT IFNULL(SUM(pm.quantity * m.price), 0)
        FROM Prescription_Medicines pm

```

```

        JOIN Medicines m ON m.medicine_id = pm.medicine_id
        WHERE pm.prescription_id = NEW.prescription_id
    ) + tax - discount
WHERE prescription_id = NEW.prescription_id;

IF OLD.prescription_id <> NEW.prescription_id THEN
    UPDATE Bills
    SET total = (
        SELECT IFNULL(SUM(pm.quantity * m.price), 0)
        FROM Prescription_Medicines pm
        JOIN Medicines m ON m.medicine_id = pm.medicine_id
        WHERE pm.prescription_id = OLD.prescription_id
    ) + tax - discount
    WHERE prescription_id = OLD.prescription_id;
END IF;
END//

CREATE TRIGGER prescription_medicine_after_delete
AFTER DELETE ON Prescription_Medicines
FOR EACH ROW
BEGIN
    UPDATE Bills
    SET total = (
        SELECT IFNULL(SUM(pm.quantity * m.price), 0)
        FROM Prescription_Medicines pm
        JOIN Medicines m ON m.medicine_id = pm.medicine_id
        WHERE pm.prescription_id = OLD.prescription_id
    ) + tax - discount
    WHERE prescription_id = OLD.prescription_id;
END//

DELIMITER ;

```

Trigger 1 - bills_before_insert:

This trigger automatically adds the total of the bill before inserting a new bill record. It adds tax to the subtotal (quantity*price of all medicines in the prescription), then subtract discount. If there are no medicines, the subtotal is zero by default.

Trigger 2 - bills_before_update:

This trigger will re-calculate the total amount of the bill every time an existing bill is updated with tax or discount values. So when you change the subtotal, tax, or discount, the total is always correct without any additional work.

Trigger 3 – prescription_medicine_after_insert:

This trigger updates the total bill for a prescription when a new medicine is added. Whenever a medicine is prescribed the system automatically re-calculates the bill for the new medicine cost.

Trigger 4 - prescription_medicine_after_update:

This trigger handles changes to prescription medicines. If the quantity of a medicine is changed, the total bill gets updated automatically. If a medicine is transferred to some other prescription, both the old and the new prescription bills are updated.

Trigger 5 - prescription_medicine_after_delete:

This trigger updates the total bill when a medicine is removed from the prescription. If a medicine is deleted, the cost of that medicine is automatically deducted from the bill.

Queries:

```
USE medicine_inventory_db;
```

```
SELECT doctor_name
```

```
FROM Doctors
```

```
WHERE specialization = 'Heart Disease';
```

```
SELECT disease_name
```

```
FROM Diseases
```

```
WHERE disease_type = 'Deficiency'
```

```
ORDER BY disease_name;
```

```
SELECT m.medicine_name, SUM(pm.quantity) AS total_sold
FROM Prescription_Medicines pm
JOIN Prescriptions pr ON pm.prescription_id = pr.prescription_id
JOIN Medicines m ON pm.medicine_id = m.medicine_id
WHERE YEAR(pr.prescription_date) = 2023
GROUP BY m.medicine_id
ORDER BY total_sold DESC;
```

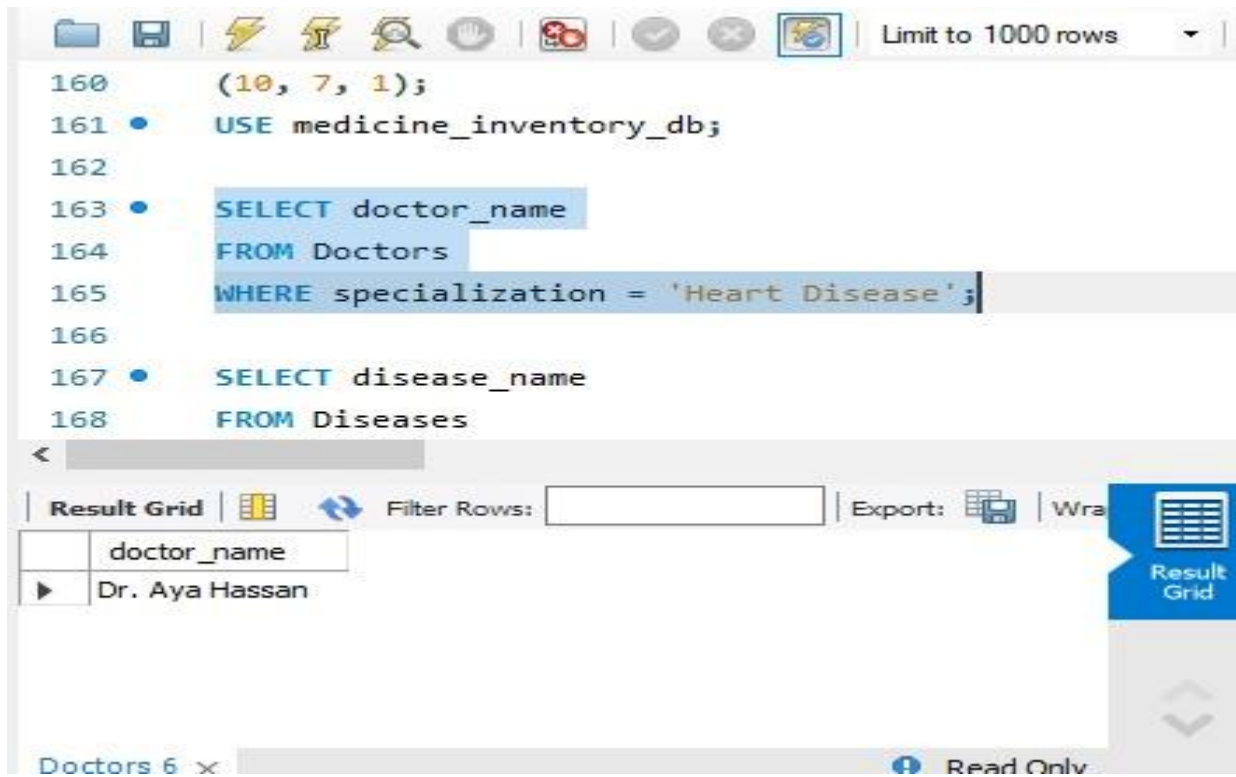
```
SELECT pr.prescription_id, pr.patient_name, d.doctor_name, pr.prescription_date
FROM Prescriptions pr
JOIN Doctors d ON pr.doctor_id = d.doctor_id
ORDER BY pr.prescription_date DESC;
```

```
SELECT b.bill_id, b.prescription_id, b.total
FROM Bills b
ORDER BY b.total DESC
LIMIT 5;
```

```
UPDATE Bills
SET discount = 5.00
WHERE prescription_id = 4;
```

```
DELETE FROM Prescription_Medicines
WHERE prescription_medicine_id = 1;
```

Query 1:



The screenshot shows a SQL query editor with the following code:

```
160      (10, 7, 1);  
161 •    USE medicine_inventory_db;  
162  
163 •    SELECT doctor_name  
164      FROM Doctors  
165     WHERE specialization = 'Heart Disease';  
166  
167 •    SELECT disease_name  
168      FROM Diseases
```

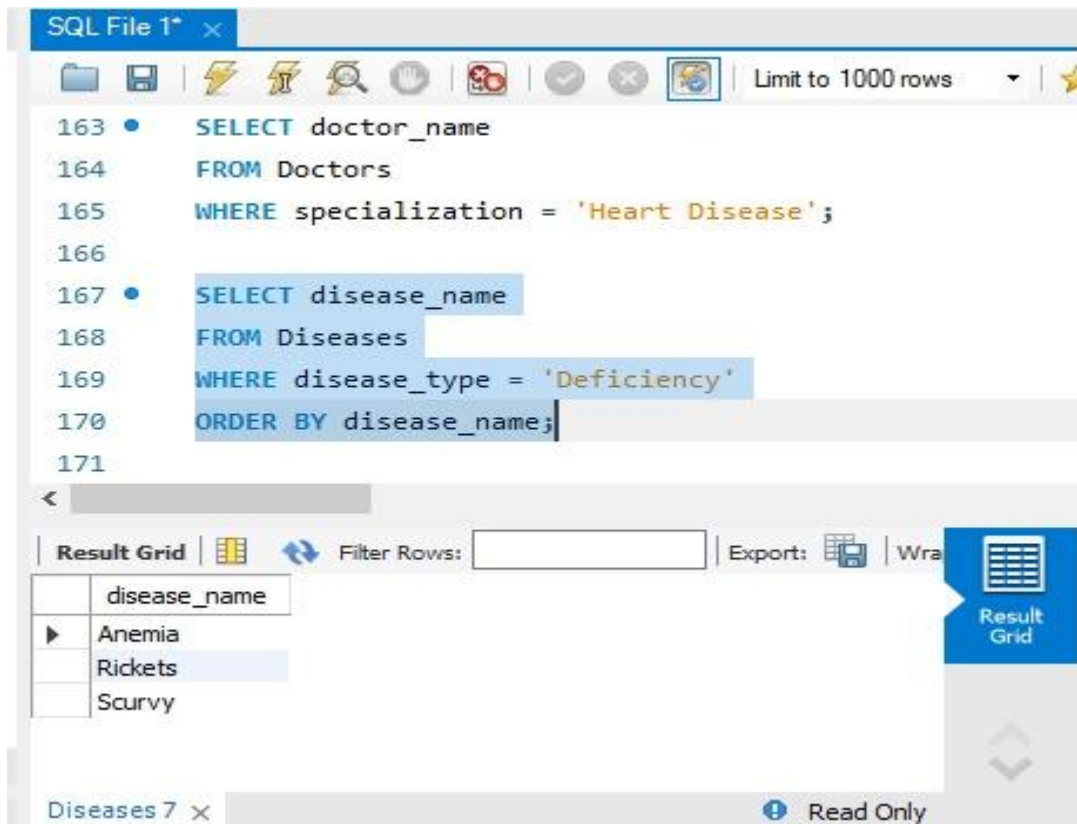
Below the query editor, the 'Result Grid' is visible, showing the results of the first query:

doctor_name
Dr. Aya Hassan

The interface includes a toolbar at the top with various icons and a 'Limit to 1000 rows' dropdown. The bottom status bar shows 'Doctors 6 x' and 'Read Only'.

This query retrieves the names of all doctors who specialize in Heart Disease.

Query 2:



```
163 • SELECT doctor_name
164 FROM Doctors
165 WHERE specialization = 'Heart Disease';
166
167 • SELECT disease_name
168 FROM Diseases
169 WHERE disease_type = 'Deficiency'
170 ORDER BY disease_name;
171
```

Result Grid

	disease_name
▶	Anemia
	Rickets
	Scurvy

Diseases 7 x Read Only

This query lists all diseases classified as 'Deficiency' and sorts them alphabetically by name.

Query 3:

The screenshot displays the MySQL Workbench interface. The 'Query' tab is active, showing a SQL query that calculates the total quantity sold for each medicine in 2023. The query is as follows:

```
10 ORDER BY disease_name;
11
12 SELECT m.medicine_name, SUM(pm.quantity) AS total_sold
13 FROM Prescription_Medicines pm
14 JOIN Prescriptions pr ON pm.prescription_id = pr.prescription_id
15 JOIN Medicines m ON pm.medicine_id = m.medicine_id
16 WHERE YEAR(pr.prescription_date) = 2023
17 GROUP BY m.medicine_id
18 ORDER BY total_sold DESC;
```

The 'Result Grid' shows the following data:

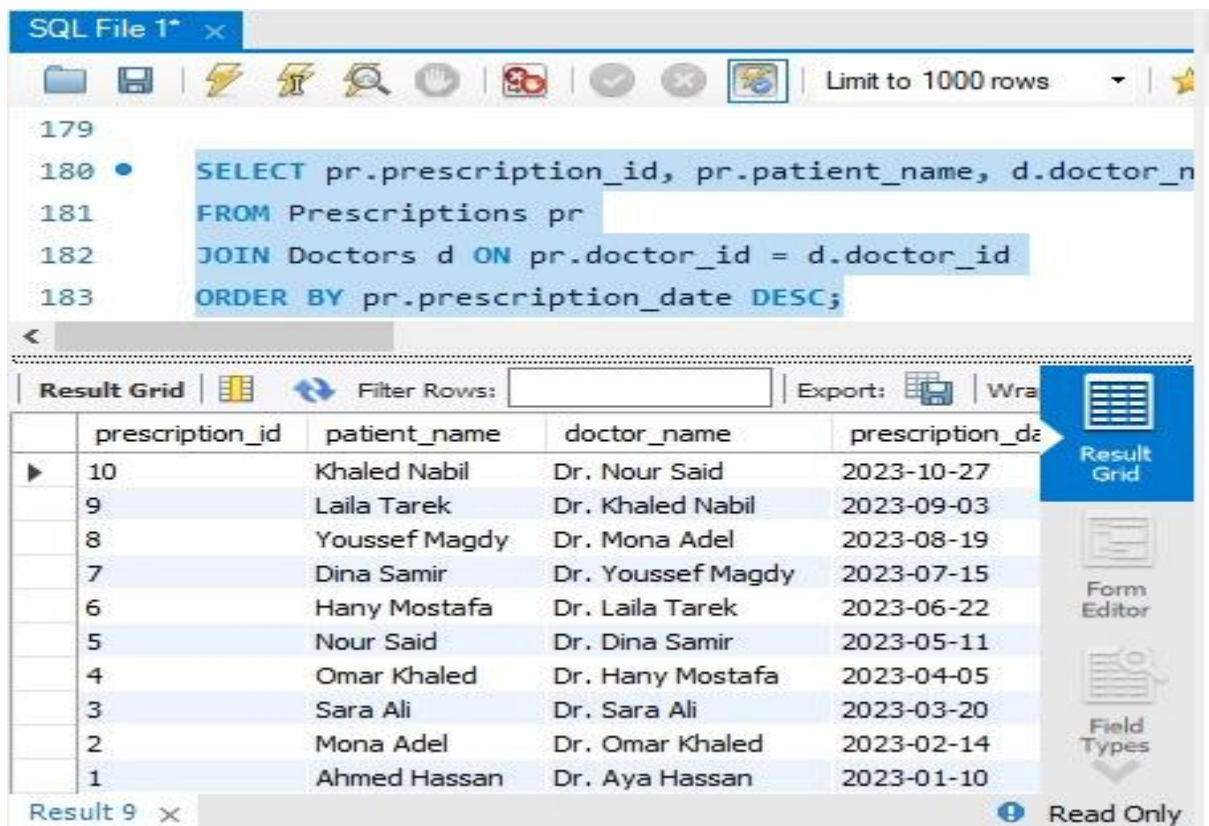
medicine_name	total_sold
Ibuprofen 400mg	3
Antihistamine 10mg	3
Aspirin 81mg	3
Paracetamol 500mg	3
Vitamin C 1000mg	2
Amoxicillin 500mg	2
Cough Syrup	2
Metformin 500mg	2
Omeprazole 20mg	2
Insulin 100IU	1

The 'Output' tab shows the execution log:

Time	Action	Message	Duration / Fetch
20:54:59	USE medicine_inventory_db	0 row(s) affected	0.000 sec
20:55:03	SELECT doctor_name FROM Doctors WHERE specialization = 'Heart Disease' LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
20:56:10	SELECT disease_name FROM Diseases WHERE disease_type = 'Deficiency' ORDER BY disease_name LIMIT ...	3 row(s) returned	0.000 sec / 0.000 sec
20:56:21	SELECT m.medicine_name, SUM(pm.quantity) AS total_sold FROM Prescription_Medicines pm JOIN Prescriptions ...	10 row(s) returned	0.016 sec / 0.000 sec

This query finds the total quantity sold for each medicine during the year 2023. It joins the Prescription_Medicines, Prescriptions, and Medicines tables to calculate the sum of quantities for each medicine, then displays the results from most sold to least sold.

Query 4:



The screenshot shows a SQL IDE window titled "SQL File 1* x". The query editor contains the following SQL code:

```
179
180 • SELECT pr.prescription_id, pr.patient_name, d.doctor_n
181 FROM Prescriptions pr
182 JOIN Doctors d ON pr.doctor_id = d.doctor_id
183 ORDER BY pr.prescription_date DESC;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has four columns: `prescription_id`, `patient_name`, `doctor_name`, and `prescription_date`. The results are sorted by `prescription_date` in descending order.

	prescription_id	patient_name	doctor_name	prescription_date
▶	10	Khaled Nabil	Dr. Nour Said	2023-10-27
	9	Laila Tarek	Dr. Khaled Nabil	2023-09-03
	8	Youssef Magdy	Dr. Mona Adel	2023-08-19
	7	Dina Samir	Dr. Youssef Magdy	2023-07-15
	6	Hany Mostafa	Dr. Laila Tarek	2023-06-22
	5	Nour Said	Dr. Dina Samir	2023-05-11
	4	Omar Khaled	Dr. Hany Mostafa	2023-04-05
	3	Sara Ali	Dr. Sara Ali	2023-03-20
	2	Mona Adel	Dr. Omar Khaled	2023-02-14
	1	Ahmed Hassan	Dr. Aya Hassan	2023-01-10

The IDE interface includes a toolbar with various icons, a "Limit to 1000 rows" dropdown, and a "Read Only" status indicator at the bottom right.

This query retrieves all prescriptions along with the patient name, the prescribing doctor's name, and the prescription date. It joins the Prescriptions and Doctors tables using the doctor-id foreign key, then sorts the results from the most recent prescription to the oldest.

Query 5:

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL query:

```
23 ORDER BY pr.prescription_date DESC;
24
25 * SELECT b.bill_id, b.prescription_id, b.total
26 FROM Bills b
27 ORDER BY b.total DESC
28 LIMIT 5;
29
30 * UPDATE Bills
31 SET discount = 5.00;
```

The 'Result Grid' shows the results of the query:

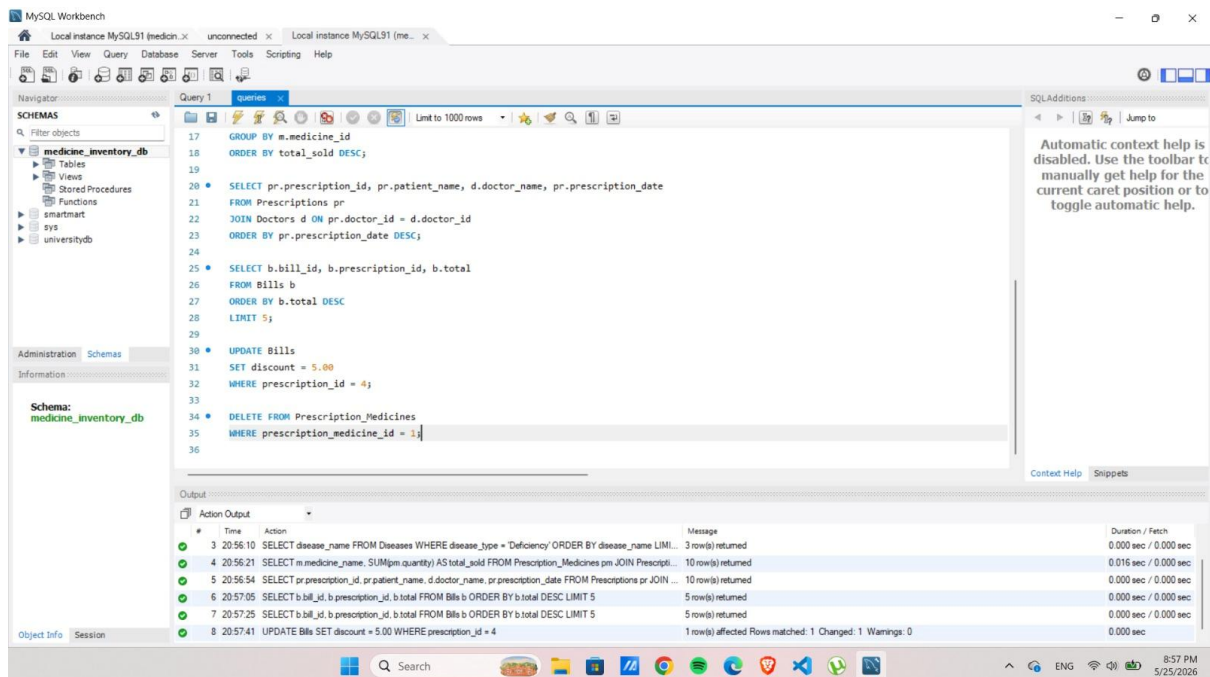
bill_id	prescription_id	total
1	1	0.00
2	2	0.00
3	3	0.00
4	4	0.00
5	5	0.00

The 'Output' tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
2	20:55:03	SELECT doctor_name FROM Doctors WHERE specialization = 'Heart Disease' LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
3	20:56:10	SELECT disease_name FROM Diseases WHERE disease_type = 'Deficiency' ORDER BY disease_name LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
4	20:56:21	SELECT m.disease_name, SUM(pn.quantity) AS total_sold FROM Prescription_Medicines pn JOIN Prescriptions pr ON pr.prescription_id = m.prescription_id	10 row(s) returned	0.016 sec / 0.000 sec
5	20:56:54	SELECT pr.prescription_id, pr.patient_name, d.doctor_name, pr.prescription_date FROM Prescriptions pr JOIN Doctors d ON pr.doctor_id = d.doctor_id	10 row(s) returned	0.000 sec / 0.000 sec
6	20:57:05	SELECT b.bill_id, b.prescription_id, b.total FROM Bills b ORDER BY b.total DESC LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
7	20:57:25	SELECT b.bill_id, b.prescription_id, b.total FROM Bills b ORDER BY b.total DESC LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec

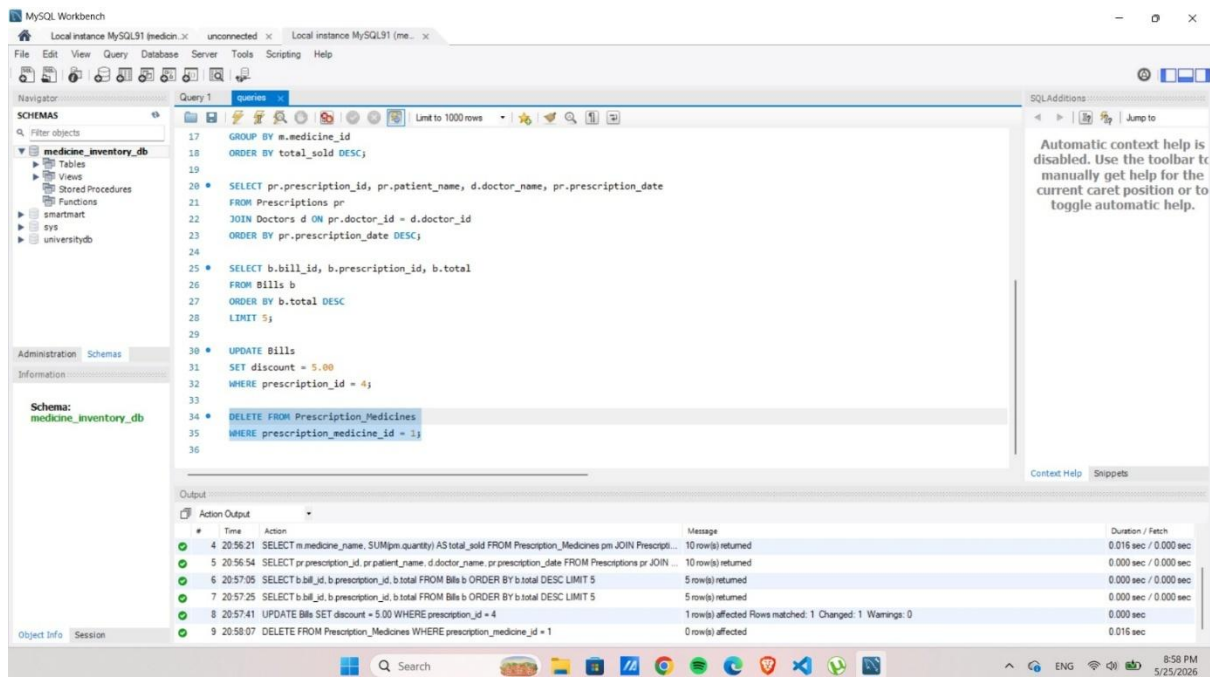
This query retrieves the five bills with the highest total amounts. It selects the bill ID, prescription ID, and total from the Bills table, then sorts the results in descending order and limits the output to only the top 5 records. NOTE: before running triggers, all bills had total =0. After triggers run, totals are calculated automatically.

Query 6:



This query applies a 5.00 discount to the bill for prescription_id=4. The total will automatically re-calculate due to the trigger.

Query 7:



This query removes the prescription medicine record with ID=1. The corresponding bill will automatically update to reflect the removed medicine.

Cardinality and Participation:

Cardinality

- Patients to Prescriptions: one-to-many.
- Prescriptions to Prescription_Items: one-to-many.
- Medicines to Prescription_Items: one-to-many.
- Prescriptions to Bills: one-to-one.

Participation

- Every prescription must belong to one patient.
- Every prescription item must reference one prescription and one medicine.
- Every bill must reference one prescription.

Indexing Considerations:

Which columns would you index to optimize lookups by patient and medicine expiry-and why?

```
CREATE INDEX idx_patient_name
```

```
ON Prescriptions(patient_name);
```

```
CREATE INDEX idx_medicine_expiry
```

```
ON Medicines(expiry_date);
```

```
SHOW INDEX FROM Prescriptions;
```

The screenshot shows a SQL IDE window with the following SQL commands executed:

```
313 ON Medicines(medicine_name);
314
315 • CREATE INDEX idx_prescription_date
316 ON Prescriptions(prescription_date);
317 • SHOW INDEX FROM Prescriptions;
318
```

The result grid displays the index information for the 'prescriptions' table:

	Table	Non_unique	Key_name	Seq_in_index	Index_type
▶	prescriptions	0	PRIMARY	1	p
	prescriptions	1	doctor_id	1	d
	prescriptions	1	idx_patient_name	1	p
	prescriptions	1	idx_prescription_date	1	p

The IDE interface includes a toolbar with icons for file operations, a 'Limit to 1000 rows' dropdown, and a 'Result Grid' button. The status bar at the bottom indicates 'Result 1' and 'Read Only'.

UI/Application:

[Home](#) [Medicines](#) [Doctors](#) [Diseases](#) [Prescriptions](#) [Bills](#)

Medicine Inventory Management System

Simple dashboard overview

Total Doctors
10

Total Diseases
10

Total Medicines
10

Total Prescriptions
10

Total Bills
10

Latest Medicines

Medicine	Expiry Date	Price
Insulin 100IU	2025-05-30	120.00
Cough Syrup	2025-06-05	55.00
Antihistamine 10mg	2025-07-25	35.00
Amoxicillin 500mg	2025-08-15	45.00
Metformin 500mg	2025-09-18	60.00

Latest Prescriptions

Prescription ID	Patient	Doctor	Date
10	Khaled Nabil	Dr. Nour Said	2023-10-27
9	Laila Tarek	Dr. Khaled Nabil	2023-09-03
8	Youssef Magdy	Dr. Mona Adel	2023-08-19
7	Dina Samir	Dr. Youssef Magdy	2023-07-15
6	Hany Mostafa	Dr. Laila Tarek	2023-06-22

Medicines

Add and search medicines

Search

Search by name

Search

Medicine Name

Manufacture Date

Expiry Date

Price

Dosage

Diseases

Anemia
Asthma
Hemophilia
Hypertension
Influenza

Add Medicine

ID	Medicine	Manufacture	Expiry	Price	Dosage	Diseases
2	Amoxicillin 500mg	2023-08-15	2025-08-15	45.00	1 capsule every 8 hours	Tuberculosis

ID	Medicine	Manufacture	Expiry	Price	Dosage	Diseases
2	Amoxicillin 500mg	2023-08-15	2025-08-15	45.00	1 capsule every 8 hours	Tuberculosis
9	Antihistamine 10mg	2023-07-25	2025-07-25	35.00	1 tablet daily	Influenza
7	Aspirin 81mg	2024-02-28	2026-02-28	18.00	1 tablet daily	Hypertension
4	Cough Syrup	2024-02-05	2025-06-05	55.00	10 ml twice daily	Influenza
3	Ibuprofen 400mg	2023-12-20	2025-12-20	30.00	1 tablet after meals	Asthma, Hypertension
10	Insulin 100IU	2023-05-30	2025-05-30	120.00	As directed	Type 2 Diabetes
6	Metformin 500mg	2023-09-18	2025-09-18	60.00	1 tablet twice daily	Type 2 Diabetes
8	Omeprazole 20mg	2023-11-02	2025-11-02	65.00	1 capsule daily	Asthma
1	Paracetamol 500mg	2024-01-10	2026-01-10	20.00	1 tablet twice daily	Influenza
5	Vitamin C 1000mg	2024-03-12	2026-03-12	25.00	1 tablet daily	Anemia, Scurvy

Doctors

Add and view doctors

Doctor Name	Qualification	Specialization
<button>Add Doctor</button>		

ID	Name	Qualification	Specialization
1	Dr. Aya Hassan	MD	Heart Disease
5	Dr. Dina Samir	MD	Gastroenterology
4	Dr. Hany Mostafa	MD	Infectious Disease
9	Dr. Khaled Nabil	MD	Dermatology
6	Dr. Laila Tarek	MD	Neurology
8	Dr. Mona Adel	PhD	Hematology
10	Dr. Nour Said	MD	Nephrology

Diseases

Add and view diseases

Disease Name	Disease Type
	Infectious
<button>Add Disease</button>	

ID	Name	Type
10	Anemia	Deficiency
7	Asthma	Non-Genetic Hereditary
6	Hemophilia	Genetic Hereditary
8	Hypertension	Non-Genetic Hereditary
1	Influenza	Infectious
4	Rickets	Deficiency
3	Scurvy	Deficiency
5	Sickle Cell Disease	Genetic Hereditary

Prescriptions

Create a prescription and add medicine items

Doctor

Dr. Aya Hassan

Patient Name

Date

mm/dd/yyyy

Create Prescription

Please fill out this field.

Prescription

#10 - Khaled Nabil

Medicine

Amoxicillin 500mg

Quantity

Add Medicine Item

ID	Patient	Doctor	Date
10	Khaled Nabil	Dr. Nour Said	2023-10-27
9	Laila Tarek	Dr. Khaled Nabil	2023-09-03
8	Youssef Magdy	Dr. Mona Adel	2023-08-19
7	Dina Samir	Dr. Youssef Magdy	2023-07-15

Bills

[View calculated totals](#)

Bill ID	Prescription	Patient	Doctor	Tax	Discount	Total
4	4	Omar Khaled	Dr. Hany Mostafa	6.25	5.00	126.25
1	1	Ahmed Hassan	Dr. Aya Hassan	3.25	0.00	0.00
2	2	Mona Adel	Dr. Omar Khaled	5.00	0.00	0.00
3	3	Sara Ali	Dr. Sara Ali	4.75	0.00	0.00
5	5	Nour Said	Dr. Dina Samir	2.80	0.00	0.00
6	6	Hany Mostafa	Dr. Laila Tarek	7.25	10.00	0.00
7	7	Dina Samir	Dr. Youssef Magdy	3.75	0.00	0.00
8	8	Youssef Magdy	Dr. Mona Adel	6.25	0.00	0.00
9	9	Laila Tarek	Dr. Khaled Nabil	6.25	5.00	0.00
10	10	Khaled Nabil	Dr. Nour Said	2.90	0.00	0.00